SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Project: TaskFlow - MERN Task Management System *ICT Academy of Kerala (ICTAK) | Full Stack Development*

Student & Submission Details

- **Student Name:** Reo Varghese
- **Batch:** CS D
- **Roll Number:** 21
- **Student Registration (SR) Number:** 225783
- **Project Name:** TaskFlow - Task & Productivity Management System
- **GitHub Repository:** https://github.com/Reo257/taskflow-mern
- **Live Frontend (Vercel):** https://taskflow-mern-tau.vercel.app
- **Live Backend API (Render):** https://taskflow-mern-nfex.onrender.com
- **Database Service:** MongoDB Atlas (AWS M0 Cluster)

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) establishes the complete functional and non-functional requirements for TaskFlow, a modern full-stack web application designed for personal and academic task management. It defines the architectural patterns, security controls, data schemas, and API contracts conforming to ICTAK standards.

1.2 Scope

TaskFlow is engineered with the MERN technology stack (MongoDB, Express.js, React 18 with Vite, and Node.js). The system features JWT authentication, bcrypt password encryption, responsive SPA interface, strict user data isolation, and comprehensive task CRUD functionality.

2. Overall System Description

2.1 Product Perspective

TaskFlow is an independent, 3-tier cloud application consisting of a React Single-Page Application (SPA) hosted on Vercel, a Node/Express REST API hosted on Render, and a distributed cloud database on MongoDB Atlas.

2.2 User Classes and Roles

• **Regular User:** Can register, authenticate, create, filter, paginate, update, and delete personal tasks. Cannot access records belonging to other users.

• **System Administrator:** Has global inspection permissions and access to aggregate system health telemetry.

3. Functional Requirements

• **FR-01 User Registration:** System accepts Name, valid Email, and Password (>= 6 chars). Hashes password using bcrypt (10 salt rounds) and rejects duplicates with 409 Conflict.

• **FR-02 User Authentication:** Authenticates email and password. Generates signed JSON Web Token (JWT) with 7-day expiry upon successful verification.

• **FR-03 Session Persistence:** React client maintains session via localStorage and automatically injects Bearer token into Authorization headers of API calls.

• **FR-04 Task Creation:** Allows authenticated users to create tasks with Title, Description, Status, Priority, Due Date, and Tag array. Validates non-empty title.

• **FR-05 Data Isolation & Ownership:** Enforces server-side data isolation. Users only see and modify tasks where task.user matches the authenticated user ID. Cross-user access returns 403 Forbidden.

• **FR-06 Filtering & Search:** Provides real-time substring search across title/description and status/priority query filters.

• **FR-07 Server-Side Pagination:** Implements page and limit parameters with total count and totalPages metadata for optimal performance.

• **FR-08 Real-Time Metrics:** Aggregates user task workload and displays counters for Total, Pending, In Progress, and Completed tasks.

• **FR-09 4-State UI Compliance:** Every screen reliably renders Loading (animated spinner), Success (data grid), Empty (friendly graphic & CTA), and Error (alert & retry button).

• **FR-10 Centralized Error Handling:** Express error middleware intercepts exceptions and returns consistent JSON errors without leaking server stack traces in production.

4. Non-Functional Requirements

• **NFR-01 Security:** Passwords encrypted via bcryptjs; protected endpoints guarded by JWT; Helmet security headers enabled; rate limiter limits auth endpoints to 100 requests per 15-minute window.

• **NFR-02 Performance:** Database utilizes composite indexes on { user: 1, createdAt: -1 } and { user: 1, status: 1 }. API response latency is under 100ms.

• **NFR-03 Usability & Responsive Design:** Mobile-first fluid layout adapting seamlessly from 320px mobile screens to large desktop monitors. Accessible semantic HTML controls with clear labels.

• **NFR-04 Maintainability:** Strict separation of concerns into client/ (components, pages, services, context) and server/ (controllers, models, middleware, routes, config).

5. API Specification & Contract

Method & Path	Auth	Purpose	Success	Errors
GET /api/health	No	Health check	200 OK	500
POST /api/auth/register	No	User signup	201 Created	400, 409
POST /api/auth/login	No	User login	200 OK	400, 401
GET /api/auth/me	Bearer	Current profile	200 OK	401
GET /api/tasks	Bearer	List & paginate tasks	200 OK	401
POST /api/tasks	Bearer	Create task	201 Created	400, 401
PUT /api/tasks/:id	Bearer	Update task	200 OK	400, 403, 404
DELETE /api/tasks/:id	Bearer	Delete task	200 OK	401, 403, 404

6. Verification & Test Evidence

The system was subjected to 19 automated integration test cases using Node.js and an in-memory database environment. 100% of test cases passed covering user registration, duplicate prevention, bad credentials, task CRUD, data isolation, and unauthorized access interception.